파이썬 데이터 구조:
리스트, 튜플, 딕셔너리, 집합

이청용 교수
leecy@hansung.ac.kr

강의개요

- 리스트(List)
  - 리스트 개요
  - 특정 위치 값 출력
  - 리스트 수정
  - 특정 위치 값 삭제
  - 리스트 확인
  - 리스트 정렬
- 튜플(Tuple)
  - 튜플 개요
  - 튜플 언패킹(Tuple Unpacking)
  - 튜플과 리스트 비교
- 딕셔너리(Dictionary)
  - 딕셔너리 개요
  - 딕셔너리 수정
  - 항목 삭제
  - 항목 확인
  - 항목 출력
- 집합(Set)
  - 집합 개요
  - 집합 계산
  - 교집합 계산
  - 합집합 계산
  - 차집합 계산

리스트(List)

리스트 개요

- **리스트 개념**
  - 데이터 순차적 파악에 유용하며, 데이터 순서 변경 가능
  - 특정 위치 값 추가, 삭제, 수정 기능
  - 동일한 값 여러 번 출현 허용
- **리스트 생성 방법**
  - 대괄호 [ ] 안에 콤마로 구분된 값을 지정
  - 빈 대괄호 [ ] 혹은 list( ) 함수로 빈 리스트 생성 가능

리스트 개요

```python
# 숫자 리스트
numbers = [10, 20, 30]
print("숫자 리스트:", numbers)

# 다양한 타입 포함
mixed = [1, 3.14, "Python", True]
print("다양한 타입 리스트:", mixed)

# 빈 리스트 생성 (대괄호)
empty1 = []
print("빈 리스트 (대괄호):", empty1)

# 빈 리스트 생성 (list 함수)
empty2 = list()
print("빈 리스트 (list 함수):", empty2)
```

Python

```
숫자 리스트: [10, 20, 30]
다양한 타입 리스트: [1, 3.14, 'Python', True]
빈 리스트 (대괄호): []
빈 리스트 (list 함수): []
```

특정 위치 값 출력

- 인덱싱(Indexing) 적용
  - 문자열과 유사하게 특정 위치 값 추출
  - 리스트의 각 요소는 0부터 시작하는 인덱스 가짐
- 음수 인덱스 활용
  - 끝에서부터 거꾸로 값을 출력 (마지막 요소는 -1)

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]

# 양의 인덱스 활용
print("세 번째 요소 (인덱스 2):", numbers[2])

# 음수 인덱스 활용
print("마지막 요소 (인덱스 -1):", numbers[-1])
```

Python

```
세 번째 요소 (인덱스 2): 30
마지막 요소 (인덱스 -1): 50
```

특정 위치 값 출력

- 슬라이스(Slice) 활용
  - 리스트의 특정 범위 값 출력
  - [start:end] 형태로 사용 (end 인덱스 미포함)
- 슬라이스 스텝(Step) 사용
  - 문자열과 유사하게 [start:end:step] 형태로 스텝 지정 가능

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]

# 기본 슬라이스 [start:end]
print("numbers[1:4] =", numbers[1:4])

# 슬라이스 스텝 [start:end:step]
print("numbers[0:5:2] =", numbers[0:5:2])
```

Python

```
numbers[1:4] = [20, 30, 40]
numbers[0:5:2] = [10, 30, 50]
```

리스트 수정

- **특정 위치 값 변경**
  - 리스트의 특정 인덱스에 새로운 값 할당하여 수정
  - 지정하는 인덱스는 반드시 유효한 범위 내여야 함

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]
print("초기 리스트:", numbers)

# 특정 위치 값 변경
numbers[2] = 99
print("세 번째 요소(인덱스 2) 수정:", numbers)
```

Python

```
초기 리스트: [10, 20, 30, 40, 50]
세 번째 요소(인덱스 2) 수정: [10, 20, 99, 40, 50]
```

리스트 수정

- append( ) 함수 활용
  - 리스트의 맨 끝에 새로운 값을 추가

```python
# 리스트 생성
numbers = [10, 20, 30]
print("초기 리스트:", numbers)

# append() 함수 활용
numbers.append(40)
print("append(40) 실행:", numbers)
```

Python

```
초기 리스트: [10, 20, 30]
append(40) 실행: [10, 20, 30, 40]
```

리스트 수정

- insert( ) 함수 활용
  - 리스트의 원하는 위치에 새로운 값을 추가
  - 지정한 위치가 유효 범위를 벗어나면 append( )처럼 리스트 끝에 값 추가

```python
# 리스트 생성
numbers = [10, 20, 30]
print("초기 리스트:", numbers)

# insert() 함수 활용
numbers.insert(1, 99)
print("insert(1, 99) 실행:", numbers)

# 범위 밖 인덱스 지정 → 맨 끝에 추가
numbers.insert(10, 50)
print("insert(10, 50) 실행:", numbers)
```

Python

```
초기 리스트: [10, 20, 30]
insert(1, 99) 실행: [10, 99, 20, 30]
insert(10, 50) 실행: [10, 99, 20, 30, 50]
```

리스트 수정

- extend( ) 함수 또는 += 연산자 활용
  - 서로 다른 리스트를 병합하여 기존 리스트 확장

```python
# 리스트 생성
numbers = [10, 20, 30]
extra = [40, 50]

# extend() 함수 활용
numbers.extend(extra)
print("extend(extra) 실행:", numbers)

# += 연산자 활용
numbers += [60, 70]
print("+= [60, 70] 실행:", numbers)
```

Python

```
extend(extra) 실행: [10, 20, 30, 40, 50]
+= [60, 70] 실행: [10, 20, 30, 40, 50, 60, 70]
```

특정 위치 값 삭제

- del 함수 활용
  - 리스트의 특정 인덱스 위치 값을 삭제
  - 삭제된 값 이후의 값들은 한 칸씩 앞으로 당겨짐

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]
print("초기 리스트:", numbers)

# del 함수 활용
del numbers[2]
print("del numbers[2] 실행:", numbers)
```

Python

```
초기 리스트: [10, 20, 30, 40, 50]
del numbers[2] 실행: [10, 20, 40, 50]
```

특정 위치 값 삭제

- remove( ) 함수 활용
  - 값의 위치를 모르는 경우, 값 자체를 지정하여 삭제
  - 동일한 값이 여러 개 있는 경우, 가장 먼저 발견되는 값 하나만 삭제

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]
print("초기 리스트:", numbers)

# remove() 함수 활용
numbers.remove(40)
print("remove(40) 실행:", numbers)
```

Python

```
초기 리스트: [10, 20, 30, 40, 50]
remove(40) 실행: [10, 20, 30, 50]
```

```python
# 동일 값이 여러 개 있을 때
dup_numbers = [1, 2, 2, 3]
dup_numbers.remove(2)
print("remove(2) 실행 (중복 값 존재):", dup_numbers)
```

Python

```
remove(2) 실행 (중복 값 존재): [1, 2, 3]
```

특정 위치 값 삭제

- pop( ) 함수 활용
  - 리스트에서 특정 위치 값을 가져오는 동시에 삭제 → 삭제된 값은 함수의 반환 값으로 얻을 수 있음
- 위치 값 미지정 시
  - 인덱스를 지정하지 않으면 리스트의 마지막 위치 값을 반환하고 삭제

```python
# 리스트 생성
numbers = [10, 20, 30, 40, 50]
print("초기 리스트:", numbers)

# 특정 위치 값 가져오고 삭제
removed = numbers.pop(2)
print("pop(2) 실행 → 반환 값:", removed, ", 리스트:", numbers)

# 위치 값 미지정 시
removed_last = numbers.pop()
print("pop() 실행 → 반환 값:", removed_last, ", 리스트:", numbers)
```

Python

```
초기 리스트: [10, 20, 30, 40, 50]
pop(2) 실행 → 반환 값: 30 , 리스트: [10, 20, 40, 50]
pop() 실행 → 반환 값: 50 , 리스트: [10, 20, 40]
```

리스트 확인

- index( ) 함수 활용
  - 리스트에 포함된 데이터의 특정 위치 값 (인덱스) 확인
  - 찾는 값이 여러 개 있을 경우, 가장 먼저 발견되는 값의 인덱스 반환
  - 찾는 값이 없으면 ValueError 발생

```python
# 리스트 생성
numbers = [10, 20, 30, 20, 40]

# index() 함수 활용
print("index(20) 실행:", numbers.index(20))
```

Python

```
index(20) 실행: 1
```

리스트 확인

- len( ) 함수 활용
  - 리스트에 포함된 데이터의 개수 (길이) 확인
- count( ) 함수 활용
  - 리스트에 포함된 특정 데이터의 빈도 (나타난 횟수) 확인

```python
# 리스트 생성
numbers = [10, 20, 20, 30, 40, 20]

# len() 함수 활용
print("리스트 길이(len):", len(numbers))

# count() 함수 활용
print("값 20의 개수(count):", numbers.count(20))
```

Python

```
리스트 길이(len): 6
값 20의 개수(count): 3
```

리스트 정렬

- sort( ) 함수 활용
  - 리스트 자체를 내부적으로 정렬 → 반환 값은 None
- 정렬 기준
  - 숫자는 오름차순 정렬, 문자열은 알파벳순 정렬
  - 내림차순 정렬: reverse = True 인자 추가

```python
# 숫자 리스트 정렬
numbers = [40, 10, 30, 20]

# sort() 함수 활용
numbers.sort()
print("sort() 실행 후 (오름차순):", numbers)

# 내림차순 정렬 (reverse=True)
numbers.sort(reverse=True)
print("sort(reverse=True) 실행 후 (내림차순):", numbers)
```

Python

```
sort() 실행 후 (오름차순): [10, 20, 30, 40]
sort(reverse=True) 실행 후 (내림차순): [40, 30, 20, 10]
```

리스트 정렬

- sorted( ) 함수 활용
  - 리스트의 정렬된 복사본을 반환

```python
fruits = ["banana", "apple", "cherry", "date"]

# 오름차순 정렬
print("sorted(fruits) 실행 결과:", sorted(fruits))

# 내림차순 정렬
print("sorted(fruits, reverse=True) 실행 결과:", sorted(fruits, reverse=True))

# 원본 리스트
print("원본 리스트:", fruits)
```

Python

```
sorted(fruits) 실행 결과: ['apple', 'banana', 'cherry', 'date']
sorted(fruits, reverse=True) 실행 결과: ['date', 'cherry', 'banana', 'apple']
원본 리스트: ['banana', 'apple', 'cherry', 'date']
```

튜플(Tuple)

튜플 개요

- **튜플 개념**
  - 리스트와 유사하나, 값 추가, 삭제, 수정 불가 (불변성)
  - "상수의 리스트"로 이해 가능
- **튜플 생성 방법**
  - 하나 이상의 데이터가 포함되는 경우, 각 데이터 뒤에 콤마(,) 추가
  - 두 개 이상 데이터 시 마지막 데이터에는 콤마 불필요
  - 괄호는 필수 아니지만, 튜플 구분을 용이하게 함

튜플 개요

```python
# 여러 자료형을 포함한 튜플
t1 = (10, 20, 30)
print("튜플 t1:", t1)

# 괄호 없이 생성 가능
t2 = 40, 50, 60
print("튜플 t2 (괄호 없이):", t2)

# 하나의 데이터만 있을 때는 반드시 콤마 필요
t3 = (70,)
print("튜플 t3 (원소 1개):", t3)

# 리스트와 달리 값 수정 불가
try:
    t1[0] = 99
except TypeError as e:
    print("튜플은 수정 불가:", e)
```

Python

```
튜플 t1: (10, 20, 30)
튜플 t2 (괄호 없이): (40, 50, 60)
튜플 t3 (원소 1개): (70,)
튜플은 수정 불가: 'tuple' object does not support item assignment
```

튜플 언패킹(Tuple Unpacking)

- **튜플 언패킹 개념**
  - 튜플의 요소를 한 번에 여러 변수에 할당하는 기능
- **변수 값 교환 활용**
  - 두 변수의 값을 교환할 때 임시 변수 없이 튜플을 사용 가능

```python
# 튜플 언패킹
t = (10, 20, 30)
a, b, c = t
print("튜플 언패킹 결과 → a:", a, ", b:", b, ", c:", c)

# 변수 값 교환
x, y = 1, 2
x, y = y, x
print("변수 값 교환 결과 → x:", x, ", y:", y)
```

Python

```
튜플 언패킹 결과 → a: 10 , b: 20 , c: 30
변수 값 교환 결과 → x: 2 , y: 1
```

튜플과 리스트

- **기능적 차이점**
  - 튜플은 append( ), insert( ) 등 리스트의 많은 함수가 없음
  - 리스트는 가변(mutable), 튜플은 불변(immutable)
- **튜플 사용 이유**
  - 공간 효율성: 리스트보다 더 적은 메모리 공간 사용
  - 데이터 안정성: 불변성으로 인해 항목 손상 염려 없음
  - 딕셔너리 키 활용: 불변 객체이므로 딕셔너리의 키로 사용 가능
  - 객체 대안: 네임드 튜플(named tuple)은 객체의 단순한 대안으로 활용
  - 함수 인자 전달: 함수의 인자들이 튜플로 전달되는 방식

튜플과 리스트

```python
# 리스트 (값 변경 가능)
lst = [10, 20, 30]
lst[0] = 99
print("리스트 값 변경 가능:", lst)

# 튜플 (값 변경 불가)
tup = (10, 20, 30)
try:
    tup[0] = 99
except TypeError as e:
    print("튜플 값 변경 불가:", e)
```

Python

```
리스트 값 변경 가능: [99, 20, 30]
튜플 값 변경 불가: 'tuple' object does not support item assignment
```

딕셔너리(Dictionary)

딕셔너리 개요

- **딕셔너리 개념**
  - 리스트와 유사하나, 순서를 따르지 않음
  - 위치 값(인덱스)으로 데이터 선택 불가
  - 값(value)에 상응하는 고유한 키(key)를 지정하여 데이터 관리
  - 키, 값 추가, 삭제, 수정 기능
- **딕셔너리 생성 방법**
  - 중괄호 { } 안에 콤마로 구분된 키:값 쌍을 지정
  - 빈 중괄호 { } 혹은 dict ( ) 함수를 사용하여 빈 딕셔너리 생성

딕셔너리 개요

```python
# 키:값 쌍으로 딕셔너리 생성
student = {"name": "Alice", "age": 21, "major": "Computer Science"}
print("딕셔너리 생성:", student)

# 빈 딕셔너리 생성 (중괄호)
empty_dict1 = {}
print("빈 딕셔너리 생성 ({}):", empty_dict1)

# 빈 딕셔너리 생성 (dict 함수)
empty_dict2 = dict()
print("빈 딕셔너리 생성 (dict()):", empty_dict2)
```

Python

```
딕셔너리 생성: {'name': 'Alice', 'age': 21, 'major': 'Computer Science'}
빈 딕셔너리 생성 ({}): {}
빈 딕셔너리 생성 (dict()): {}
```

딕셔너리 수정

- **항목 추가 및 값 변경**
  - 키(key)에 값(value)을 할당하여 항목 추가
  - 키가 이미 존재하는 경우, 해당 키의 값은 새로운 값으로 대체
  - 키가 존재하지 않는 경우, 새로운 키:값 쌍이 추가

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}
print("초기 딕셔너리:", student)

# 새로운 키:값 쌍 추가
student["major"] = "Computer Science"
print("major 키 추가:", student)

# 기존 키 값 변경
student["age"] = 22
print("age 값 변경:", student)
```

Python

```
초기 딕셔너리: {'name': 'Alice', 'age': 21}
major 키 추가: {'name': 'Alice', 'age': 21, 'major': 'Computer Science'}
age 값 변경: {'name': 'Alice', 'age': 22, 'major': 'Computer Science'}
```

딕셔너리 수정

- 키의 고유성 (uniqueness)
  - 키는 반드시 유일해야 하며, 중복되는 경우 마지막 할당 값이 적용

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}
print("초기 딕셔너리:", student)

# 키의 고유성 → 같은 키에 다시 할당 시 마지막 값 적용
student["name"] = "Bob"
print("name 키 중복 할당 결과:", student)
```

Python

```
초기 딕셔너리: {'name': 'Alice', 'age': 21}
name 키 중복 할당 결과: {'name': 'Bob', 'age': 21}
```

딕셔너리 수정

- update( ) 함수 활용
  - 하나의 딕셔너리의 키(key)와 값(value)들을 복사하여 다른 딕셔너리에 병합
  - 주로 다른 딕셔너리나 키=값 형태의 인자를 사용하여 병합

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}
print("초기 딕셔너리:", student)

# 다른 딕셔너리 병합
student.update({"age": 22, "major": "Computer Science"})
print("update(다른 딕셔너리) 실행:", student)

# 키=값 형태 인자 사용
student.update(grade="A", name="Bob")
print("update(키=값 형태) 실행:", student)
```

Python

```
초기 딕셔너리: {'name': 'Alice', 'age': 21}
update(다른 딕셔너리) 실행: {'name': 'Alice', 'age': 22, 'major': 'Computer Science'}
update(키=값 형태) 실행: {'name': 'Bob', 'age': 22, 'major': 'Computer Science', 'grade': 'A'}
```

항목 삭제

- del 함수 활용
  - 딕셔너리에 있는 특정 항목을 삭제 (대괄호에 키 지정)
  - 존재하지 않는 키를 삭제 시 KeyError 발생

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21, "major": "CS"}
print("초기 딕셔너리:", student)

# del 함수 활용 (특정 키 삭제)
del student["major"]
print("del student['major'] 실행:", student)
```

Python

```
초기 딕셔너리: {'name': 'Alice', 'age': 21, 'major': 'CS'}
del student['major'] 실행: {'name': 'Alice', 'age': 21}
```

항목 삭제

- clear( ) 함수 활용
  - 딕셔너리에 있는 모든 키(key)와 값(value)을 삭제 → 딕셔너리 자체는 유지되고 내용만 비워짐
- 빈 딕셔너리 할당
  - 기존 딕셔너리 변수에 빈 딕셔너리를 할당하여 모든 항목 삭제 가능

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21, "major": "CS"}
print("초기 딕셔너리:", student)

# clear() 함수 활용
student.clear()
print("clear() 실행:", student)

# 빈 딕셔너리 할당
student = {}
print("빈 딕셔너리 {} 할당:", student)
```

Python

```
초기 딕셔너리: {'name': 'Alice', 'age': 21, 'major': 'CS'}
clear() 실행: {}
빈 딕셔너리 {} 할당: {}
```

항목 확인

- in 연산자 활용
  - 딕셔너리에 특정 키(key)가 존재하는지 여부 확인
  - 키가 존재하면 True 값 출력, 존재하지 않으면 False 값 출력
  - not in 연산자를 사용하여 키 부재 여부 확인 가능

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}

# in 연산자
print("'name' in student:", "name" in student)
print("'grade' in student:", "grade" in student)

# not in 연산자
print("'grade' not in student:", "grade" not in student)
```

Python

```
'name' in student: True
'grade' in student: False
'grade' not in student: True
```

항목 출력

- 키(Key) 지정 방식
  - 딕셔너리에 특정 키를 지정하면 상응하는 값(value)을 직접 출력
  - 대괄호 [ ] 안에 키를 넣어 값에 접근

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}

# 키로 값에 접근
print("student['name']:", student["name"])
print("student['age']:", student["age"])
```

Python

```
student['name']: Alice
student['age']: 21
```

항목 출력

- 키(Key) 지정 방식
  - 딕셔너리에 특정 키가 존재하지 않으면 KeyError 발생

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}

# 존재하지 않는 키 접근 시 오류 발생
print("student['grade']:", student["grade"])
```

Python

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[2], line 5
      2 student = {"name": "Alice", "age": 21}
      4 # 존재하지 않는 키 접근 시 오류 발생
----> 5 print("student['grade']:", student["grade"])

KeyError: 'grade'
```

항목 출력

- get( ) 함수 활용
  - 키(key)가 존재하면 상응하는 값(value)을 출력

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}

# 키가 존재하는 경우
print("student.get('name'):", student.get("name"))
```

✓ 0.0s Python

```
student.get('name'): Alice
```

항목 출력

- get( ) 함수 활용
  - 키가 존재하지 않을 경우, 지정된 기본값을 출력 가능
  - 기본값을 지정하지 않으면 None 값을 출력하며, 결과는 아무것도 표시되지 않음

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21}

# 키가 존재하지 않는 경우
print("student.get('grade'):", student.get("grade"))

# 키가 존재하지 않는 경우(기본값 지정)
print("student.get('grade', 'N/A'):", student.get("grade", "N/A"))
```

✓ 0.0s Python

```
student.get('grade'): None
student.get('grade', 'N/A'): N/A
```

항목 출력

- keys( ) 함수 활용
  - 딕셔너리의 모든 키(key) 항목 출력
- values( ) 함수 활용
  - 딕셔너리의 모든 값(value) 항목 출력

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21, "major": "CS"}

# 모든 키 출력
print("student.keys():", student.keys())

# 모든 값 출력
print("student.values():", student.values())
```

Python

```
student.keys(): dict_keys(['name', 'age', 'major'])
student.values(): dict_values(['Alice', 21, 'CS'])
```

항목 출력

- items( ) 함수 활용
  - 딕셔너리의 모든 키(key)와 값(value) 항목을 동시에 출력
  - 각 키와 값은 (key, value) 형태의 튜플로 반환
  - dict_items 객체를 반환하며, list( )로 변환하여 사용 가능

```python
# 딕셔너리 생성
student = {"name": "Alice", "age": 21, "major": "CS"}

# 모든 키-값 쌍 출력
print("student.items():", student.items())

# list()로 변환하여 사용
print("list(student.items()):", list(student.items()))
```

Python

```
student.items(): dict_items([('name', 'Alice'), ('age', 21), ('major', 'CS')])
list(student.items()): [('name', 'Alice'), ('age', 21), ('major', 'CS')]
```

집합(Set)

집합 개요

- **집합 개념**
  - 값(value)은 없고 키(key)만 남은 딕셔너리와 유사
  - 각 요소(키)는 반드시 유일해야 함 (중복 허용 안 됨)
  - 데이터의 포함 여부 판단에 주로 사용
  - 순서가 없음
- **집합 생성 방법**
  - set( ) 함수로 리스트에서 집합 생성
  - 중괄호 { } 안에 콤마로 구분된 하나 이상의 값을 넣어서 생성
  - 빈 집합은 { } 대신 반드시 set( )으로 생성 ({ }는 빈 딕셔너리)

집합 개요

```python
# 중괄호 { } 사용
s1 = {1, 2, 3, 3}
print("집합 s1 (중복 허용 안 됨):", s1)

# set() 함수 사용
s2 = set([1, 2, 2, 3])
print("집합 s2 (리스트 변환):", s2)

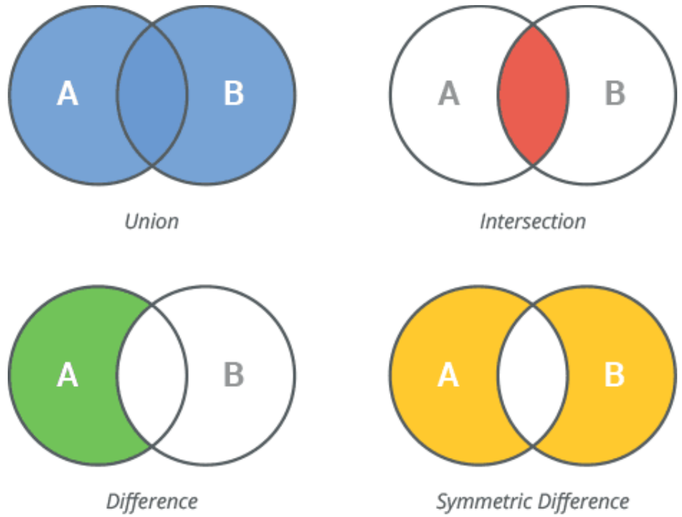
# 빈 집합 생성 (반드시 set() 사용)
empty_set = set()
print("빈 집합 생성:", empty_set)
```

Python

```
집합 s1 (중복 허용 안 됨): {1, 2, 3}
집합 s2 (리스트 변환): {1, 2, 3}
빈 집합 생성: set()
```

집합 계산

- 교집합 (Intersection)
  - 두 집합에 모두 포함되어 있는 값으로 이루어진 집합
- 합집합 (Union)
  - 첫 번째 집합에 속하거나 두 번째 집합에 속하는 값들로 이루어진 집합
  - 두 집합의 모든 고유한 요소를 포함
- 차집합(Difference)
  - 첫 번째 집합에는 있지만 두 번째 집합에는 없는 값으로 이루어진 집합
- 대칭 차집합(Symmetric Difference)
  - 두 집합 중 한쪽에만 존재하는 값들로 이루어진 집합 (양쪽에 모두 있는 값은 제외)



출처:https://www.learnbyexample.org/wp-content/uploads/python/Python-Set-Operatioons.png

교집합 계산

- 교집합(Intersection) 연산자
  - & 연산자 사용
  - intersection( ) 함수 사용

```python
# 집합 생성
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# 교집합 (공통된 요소)
print("a & b:", a & b)
print("a.intersection(b):", a.intersection(b))
```

Python

```
a & b: {3, 4}
a.intersection(b): {3, 4}
```

합집합 계산

- 합집합(Union) 연산자
  - | 연산자 사용
  - union( ) 함수 사용

```python
# 집합 생성
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# 합집합 (모든 요소, 중복 제거)
print("a | b:", a | b)
print("a.union(b):", a.union(b))
```

Python

```
a | b: {1, 2, 3, 4, 5, 6}
a.union(b): {1, 2, 3, 4, 5, 6}
```

차집합 계산

- 차집합(Difference) 연산자
  - – 연산자 사용
  - difference( ) 함수 사용

```python
# 집합 생성
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# 차집합 (a에는 있고 b에는 없는 요소)
print("a - b:", a - b)
print("a.difference(b):", a.difference(b))
```

Python

```
a - b: {1, 2}
a.difference(b): {1, 2}
```

차집합 계산

- 대칭 차집합(Symmetric Difference) 연산자
  - ^ 연산자 사용
  - symmetric_difference( ) 함수 사용

```python
# 집합 생성
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# 대칭 차집합 (두 집합 중 어느 한쪽에만 있는 요소)
print("a ^ b:", a ^ b)
print("a.symmetric_difference(b):", a.symmetric_difference(b))
```

Python

```
a ^ b: {1, 2, 5, 6}
a.symmetric_difference(b): {1, 2, 5, 6}
```

수고하셨습니다.